1. Os metódos HTTP utilizados na arquitetura RESTFUL são extremamente eficientes na manipulação de recursos, visto que a comunicação feita através desses verbos é flexível e, geralmente,causa menos sobrecarga se comparada à outras opções, como: SOAP. Esse fator torna a arquitetura em restful ser a mais usada no mundo, contribuindo para o manutenção e monitoramento do ciclo de vida de um sistema e na facil manipulação de recursos entre sistemas.

Dentre os metódos utilizados temos o GET, responsável por oferecer informações de determinado recurso, e o POST, responsável por enviar novos registros para um sistemas. Cada um desses metódos trabalham de formas diferentes, quando usamos GET não é recomendado o uso de um corpo em uma requisição, normalmente só apresenta parêmetros quando quer filtra ou buscar determinado recurso, enquanto no POST usa-se um corpo para a criação de um registro bem definido.

2. A principal vantagem do protocolo HTTP ser um protocolo stateless é que no contexto de server-side é muito vantajoso, pois favorece uma escalabilidade maior do sistema e oferece menos sobrecarga do proprio já que não precisa armazenar, dentro dos servidores, o contexto da ultima comunicação com o cliente.

Para contornar as limitações desse modelo Stateless os navegadores usam cookies e os servidores sessões, que são usados toda vez que o cliente precisa fazer uma requisição que precise de um nível de acesso dentro do servidor, para assim os servidores entenderem quem é quem dentro de um sistema e manterem também o estado da comunicação.

3. As familias que indicam sucesso, falha do cliente e falha do servidor em uma requisição são, respectivamente: 2xx, 4xx e 5xx.

O codigo 200 OK indica o sucesso de uma requisição, ou seja, ela foi concluída de forma correta sem nenhuma problema, 404 Not Found indica um erro do cliente ao tentar localizar um recurso em um servidor, muito provavelmente porque esse recruso não existe mais, 500 Internal Server Error, como o proprio nome diz, indica um erro interno no servidor.

4. O envio de dados através de uma requisição POST em http simples não garante segurança, pois a interceptação e comprometimento desses dados é um ponto de vista totalemente realista. Para resolver isso surge o https, que é o protocolo http com uma camada de criptografia/TLS, ou, transport layer security. Essa camada de criptografia é responsável por garantir a segurança de dados enviados em, por exemplo, requisições post. O servidor troca chaves com o cliente logo no inicio da sessão(handShake) e garante a segurança dos dados.

5. Esses formatos utilizados pelas APIs como o json ou xml, são formatos de serialização de recursos, basicamente, esses recursos são transformados em texto e o inverso também acontece(desserialização), o que viabiliza a comunicação entre sistemas heterogêneos através de uma API. O numero excessivo de requisições simultâneas pode causar um impacto no desempenho do sistema e até mesmo sobrecargas, uma vez que, a largura de banda é limitada e, também, há consumo de poder de processamento. Isso se agrava mais ainda caso não seja uma comunicação via http simples, em https o handshake TLS aumenta a latência, consumo de banda e processamento deixando mais propenço a sobrecargas.

6. Quando falamos de Arquiteturas de software, falamos de como os componentes de um sistema interagem e se comunicam entre si, essa interação pode ocorre de varias maneiras, sendo duas delas: baseada em eventos e centrada em dados. Na arquitetura baseada em eventos os componentes se comunicam atravás da publicação de eventos, podendo ser intermediada ou não por um broker, esse evento interaje com os demais componentes necessarios de um sistema atualizando informação, gerando registros e etc. Já na arquitetura centrada em dados, temos um repositório central de dados compartilhado entre os componentes, ele garante a comunicação entre as partes do sistema pela consulta, publicação ou compartilhamento de dados.

Um exemplo de cada arquitetura seria em um cenário comercial, que possuo os componentes de catalogo de venda, estoque e envio. Na arquitetura baseada em eventos aconteceria o seguinte: Quando um cliente fizesse uma compra, o catalogo de venda publicaria um evento, esse evento atualizaria o estoque e emitiria o envio do produto, assim, interajindo com os outros componentes do sistama. Já na arquitetura centrada em dados, quando o cliente realizasse uma compra, o componente de catologo de vendas se comunicaria com o banco de dados central e atuzalizaria/emitiria as informação necessarias, assim, afetando os demais componentes, esroque e envio, que também utilizam esse banco de dados central.

7. Quando falamos das arquiteturas de sistemas distribuídos, os principais modelos que vem em mente são o cliente-servidor e peer-to-peer(P2P). No modelo cliente-servidor à uma relação fixa de que o servidor é quem provê recursos solicitados pelo cliente, já em P2P ocorre de outra maneira a comunicação entre os sistemas: Cada nó do sistemas é tanto porvedor de recursos com consumidor. Pelo ponto de vista da escalabilidade, o modelo P2P tem um potencial muito superior ao cliente-servidor, pois neste modelo não existe nenhuma centralização de recursos ou usuarios, algo que também contribui na conscistência dos serviços, já que, não possui um ponto único de falha, diferentemente de um servidor central.

8. O modelo cliente-servidor não é uma relação extrita entre varios cliente e um unico serviço, esses servidores fornecedores de recursos também podem ser clientes de outros serviços. Seguindo está logica esses serviços podem apresentar níveis partindo da requisição do usuario final. Na arquitetura

3-tier, temos uma estrutura organizada que começa pela interface, camada de apresentação, que é onde o usuario final interaje com o sistema, realizando requisições. Em seguida, vem a camada de aplicação, é nela que o sistema aplica a lógica e as regras de negócio, é o intermédio entre a interface e o banco de dados. E por fim vem a camada de dados, ela é a responsável por armazenar e gerenciar os dados de um sistema, o usuario final nunca interaje diretamente com essa camada nesse modelo de arquitetura. Essa separação lógica e física das estruturas possue grande vantagem em escalabilidade, distribuição da força de processamento, manutenção e sobrecarga, quando comparamos com uma arquitetura monolítica de 2 níveis, nessa arquitetura o usúario roda a interface, nível 1, juntamente com toda a lógica e as regras de negócio, balanceando a carga como estratégia, essa camada da interface é quem vai fazer, também, a conexão com o banco de dados centralizado, que é o nível 2.

9. Servidores em cluster são um conjunto de duas ou mais maquinas que são conectados e gerênciados como um servidor unico, esse modelo se destacada devido a metrica de escalabilidade horizontal. As maquinas que se unem ao conjunto são chamados de nós, esses nós fazem o balanceamento de carga de processamento, garantindo um alto desempenho e segurança com a possibilidade de um nó assumir a operação de um nó que deu algum tipo de problema(failover). Essas propriedades garantem uma alta disponibilidade, eliminando, na maioria das vezes, pontos únicos de falha, e desempenho do sistema, já que há uma distribuição das requisições pelos nós.

10. O uso de um banco de dados centralizado não seria uma boa escolha para esse tipo de serviço, pois haveria uma alta conscentração de dados, o que faria esse sistema de mapeamente de ips lento e muito vulnerável à sobrecargas. Além de que, não seria nada facil organizar e gerênciar um banco de dados desse porte, seria caro e exigiria muito trabalho. Por isso, o DNS utiliza uma hierarquia, que são: Os servidores raiz, que direcionam as consultas aos servidores de TLD, que indicam os servidores autoritativos responsáveis pelo domínio e por fornecer seu endereço IP correspondente.

11. Quando utilizamos estruturas de dados na memória para armazenar informações de uma API, podemos ter problemas como a perda dos dados quando o servidor é desligado ou reiniciado, além de dificuldades para compartilhar esses dados entre diferentes instâncias da aplicação. Para corrigir esses problemas, podemos utilizar um banco de dados, que permite armazenar as informações de forma persistente, mantendo os dados mesmo após o encerramento do servidor.